

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	1 de 24
DACTA SAC			

1. Introducción	3
1.1. Objetivo	3
1.2. Alcance	3
2. Directorio de desarrollo	4
3. Diseño de la Arquitectura	5
3.1. Cliente-Servidor (Power Builder)	5
3.2. Intengrens Framework Web (C#, JavaScript – Framework Antiguo)	6
3.3. Intengrens Framework Web y Otros portales Web (React TS – Framework Nuevo)	6
3.4. Microservicios (Node TypeScript)	7
3.5. Microservicios (FastApi Python)	8
3.6. Movil (Java)	9
3.7. Movil (React TS native)	9
4. Declaraciones	10
4.1. Abreviaturas de variable	10
4.2. Acciones	10
4.3. Tipo de datos	11
5. Nomenclaturas Intengrens	11
5.1. Módulos y Submódulos	11
5.1.1. Módulos (MMM)	12
5.1.2. Submódulos Intengrens (MMM)	12
5.1.2.1. Submódulos de Comercialización (CMR)	12
5.1.2.2. Submódulos de Finanzas (FIN)	12
5.1.2.3. Submódulos de Logística (LOG)	12
5.1.2.4. Submódulos de Recursos Humanos (RH)	13
5.2. Nombre de clases, archivos o librerías	13
5.2.1. Cliente-Servidor (Power Builder) (Librería)	13
5.2.2. Web (C#, JavaScript, Typescript, Python, Java)	13
5.2.3. Móvil (Java) (clase)	14
5.2.4. Movil (React TS native)	14
5.3. Nombre de variables	14
5.4. Nombre de métodos o procedimientos	15
5.5. Nombre de funciones	15
5.6. Nombre de parámetros	16

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	2 de 24
DACTA SAC			

5.7. Nombre de formularios, ventana, window	16
5.7.1. Cliente-Servidor (Power Builder).....	16
5.7.2. Web (C#, JavaScript).....	16
5.7.3. Móvil (Java)	17
5.8. Nombre de controles (excluye componentes react).....	17
6. Convenciones de código	18
6.1. Diseño del formato del código	18
6.2. Comentarios	19
7. Políticas para nuevos desarrollos.....	19
7.1. Pruebas unitarias.....	19
7.2. Programación de Logs.....	20
7.3. Documentación de los proyectos	20
7.4. Sobre el uso del Git para DevOps	20
8. Buenas prácticas de programación.....	20
9. Manejo de Excepciones	22
10. Mensajes de alerta.....	22
10.1. Cliente-Servidor (Power Buiden)	22
10.2. Web (JavaScript)	23
10.3. Móvil (Java)	23
11. Seguridad de la información	23
11.1. Normativa de Seguridad de la Información.....	23
12. Control de cambios.....	24

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	3 de 24
DACTA SAC			

1. Introducción

El presente documento cubre los aspectos relevantes a tomar en cuenta en el proceso de desarrollo de software. La ejecución de las directivas expuestas es responsabilidad directa del personal de desarrollo de sistemas de DACTA. Toda persona deberá tener en cuenta el cumplimiento de estas directivas.

1.1. Objetivo

El objetivo del presente documento es lograr establecer los lineamientos que se deben seguir para el desarrollo de software que vayan a ser implementados, y que a su vez busca lograr un efecto positivo en:

- Asegurar la calidad del producto a desarrollar
- Permitir facilidad en mantenimiento del mismo
- Estandarizar la forma de trabajo en lo que respecta a aplicaciones
- Evitar errores, especialmente aquellos difíciles de detectar
- Simplificar la detección de errores y correcciones de bugs
- Simplificar el monitoreo de las ejecuciones
- Confiabilidad en el código generado

Estos objetivos van alineados con el uso de la metodología orientada a objetos y servicios.

1.2. Alcance

Todas las aplicaciones desarrolladas en DACTA trabajarán de acuerdo a los estándares establecidos en el presente documento.

Se abarca los conceptos de nomenclaturas, definición de variables, métodos, funciones, manejo de errores y comentarios, así como buenas prácticas en la programación.

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	4 de 24
DACTA SAC			

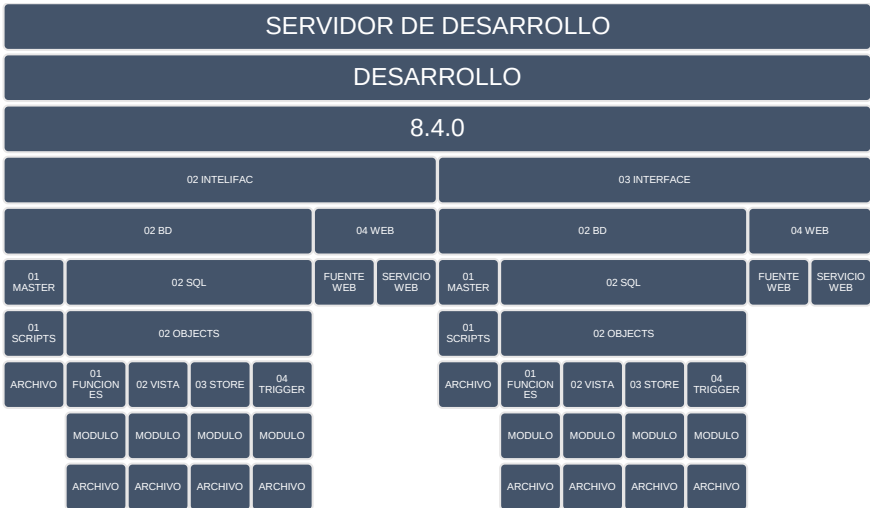
2. Directorio de desarrollo

Los desarrollos se alojan en un sitio específico para evitar que existan múltiples versiones. La carpeta donde se encuentran los archivos fuentes variará dependiendo del lenguaje de programación a desarrollar. Tomar en cuenta de que cada recuadro corresponde a un directorio.

- Fuentes de solución Cliente-Servidor:

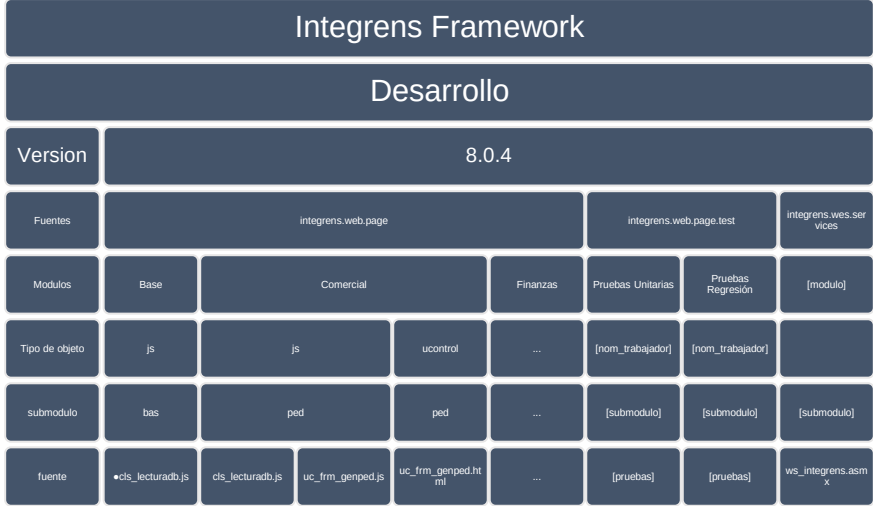


- Fuentes de solución de Facturación Electrónica:

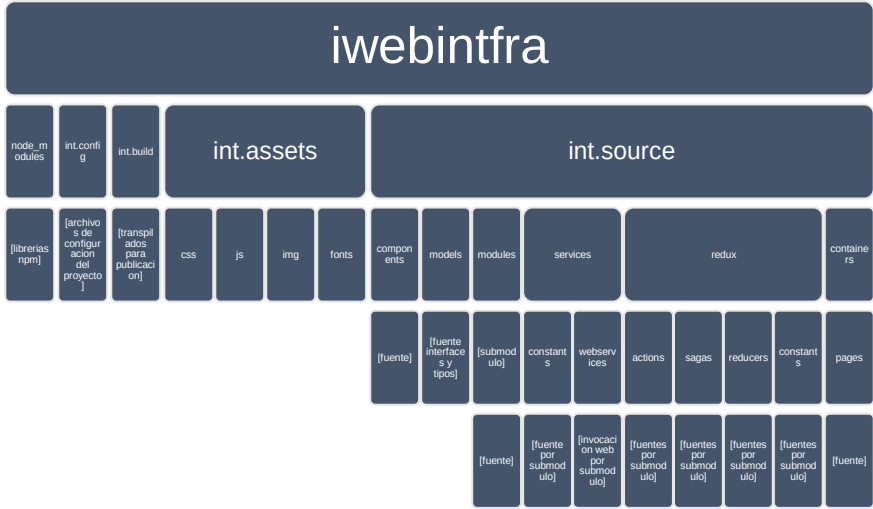


	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	6 de 24
DACTA SAC			

3.2. Intengrens Framework Web (C#, JavaScript – Framework Antiguo)

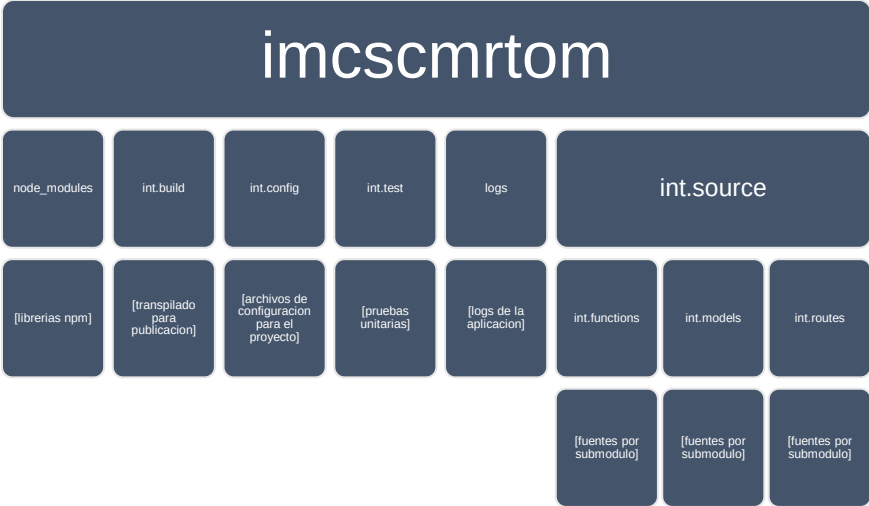


3.3. Intengrens Framework Web y Otros portales Web (React TS – Framework Nuevo)



 DACTA SAC	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	7 de 24

3.4. Microservicios (Node TypeScript)

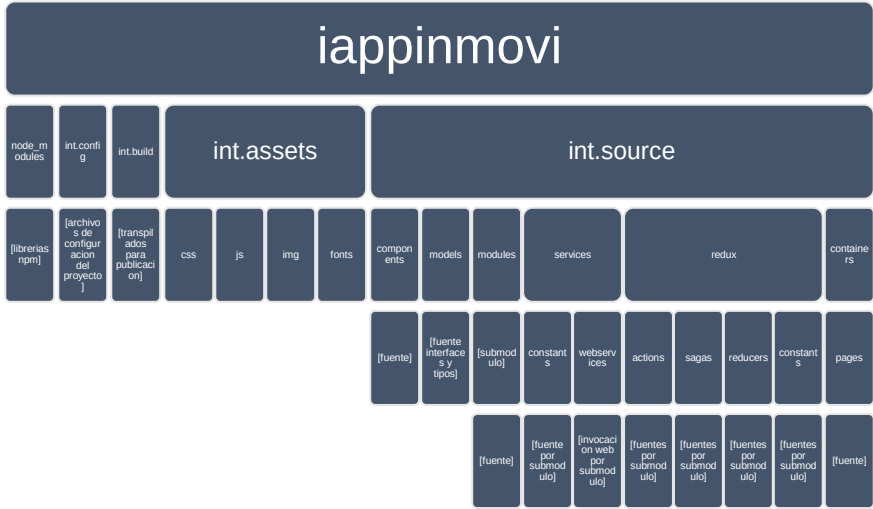


	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	9 de 24
DACTA SAC			

3.6. Movil (Java)



3.7. Movil (React TS native)



	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	10 de 24
DACTA SAC			

4. Declaraciones

Existe una variante de estilos o convenciones de estándares para nombrar e identificar a los elementos de un proyecto de desarrollo de software. Para nuestro caso utilizaremos la siguiente:

- **Notación de Serpiente (Snake Case):** Todas las palabras se escriben en minúsculas separadas por guiones bajo, ejemplo: codigo_cliente_avanzado.

Los identificadores brindan un nombre a la aplicación, clases, métodos, variables, tipos, etc. Colaboran en la auto documentación del desarrollo de software, aportando información sobre el cometido que llevan a cabo (el qué y no el cómo). Para elegir los nombres se deben seguir las siguientes recomendaciones generales.

- Elegir el nombre comprensible y en relación con la tarea que corresponda al objeto nombrado.

Utilice palabras abreviadas adoptando muchas formas, en consecuencia, pueden resultar confusas. Elija las abreviaturas adecuadamente.

4.1. Abreviaturas de variable

La abreviatura de las variables está definida según su alcance:

Abreviatura	Alcance
g	Global, a nivel de todo el proyecto.
l	Local, a nivel del método, función o evento.
i	Instancia, a nivel de objeto, ventana/formulario.
a	Argumento, a nivel del método, función o evento.

```
cl_prueba.cs/cl_prueba.java
public class cl_prueba {

    string gs_fecprc;

    public cl_prueba() {
        --
    }

    function uf_metodo(string as_fecprc) {
        string ls_fecprc;
        ls_fecprc = gs_fecprc;
        ls_fecprc = as_fecprc;
    }
}
```

4.2. Acciones

La definición de las acciones de los métodos y/o funciones está guiado por las siguientes reglas:

Abreviatura	Acción
-------------	--------

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	11 de 24
DACTA SAC			

ins	Insertar
upd	Actualizar
del	Eliminar
con	Consultar

4.3. Tipo de datos

Las variables deben especificar un prefijo obligatorio para indicar el tipo de dato al cual pertenecen. No están excluidos de la regla las variables temporales como acumuladores, contadores, temporales.

Abrev	Tipo de dato	C#	Java	Python	Type Script	Java script
s	String, Cadena	✓	✓	☐	✓	☐
i	Int, Integer, Entero	✓	✓			☐
b	Bool, Boolean, Boleano	✓	✓	☐	✓	☐
dc	Decimal/Double	✓	✓	☐		☐
d	Date	✓	✓	☐	✓	☐
dt	DateTime	✓	✓			
ar	Array	✓	✓	☐	✓	☐
li	List	✓	✓	☐		
c	Char	✓	✓			
bt	Byte	✓	✓			
ob	Object	✓	✓	☐	✓	☐
bf	Buffer	✓	✓	☐	✓	☐
dy	Dictionary	✓		☐		
hs	HashSet	✓	✓			
st	Stream	✓	✓			
fl	File	✓	✓	☐	✓	☐
en	Enum	✓	✓	☐	✓	☐
v	Void				✓	☐
th	Thread	✓	✓	☐		
rx	Regex	✓	✓	☐	✓	☐
dt	DataTable	✓				
dc	Data Column	✓				
dr	Data Row	✓				
da	Data Adapter	✓				

(✓) - Usan esos tipos de variable.

(☐) - No son tipados explícitamente. Pero implícitamente usa dichos tipos de variables.

5. Nomenclaturas Integrens

5.1. Módulos y Submódulos

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
DACTA SAC		Página	12 de 24

5.1.1. Módulos (MMM)

Abreviatura	Sistema
ACC	Control de activos en comodato
CFG	Configuración
CMR	Comercialización
CRM	CRM
FIN	Finanzas
LOG	Logística
MAN	Mantenimiento
PRD	Producción
RH	Recursos Humanos
SYS	Sistema
SEG	Seguridad

5.1.2. Submódulos Integrens (MMM)

5.1.2.1. Submódulos de Comercialización (CMR)

Abreviatura	Submódulo
CMR	EST Estadísticas de ventas
	PED Pedidos
	PLV Planeamiento de ventas
	TOM Tomapedidos
	GEC Información Geográfica de Clientes y Rutas de Ventas

5.1.2.2. Submódulos de Finanzas (FIN)

Abreviatura	Submódulo
FIN	BUD Presupuestos
	CON Contabilidad
	COS Costos
	CXC Créditos y cobranzas
	CXP Cuentas por pagar
	TES Caja y tesorería

5.1.2.3. Submódulos de Logística (LOG)

Abreviatura	Submódulo
LOG	ALM Almacenes
	CIM Importaciones
	CMN Compras
	CTZ Cotizaciones
	DIS Distribución
	REQ Requisiciones
	GEL Información Geográfica de Clientes y Rutas de Distribución

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	13 de 24
DACTA SAC			

5.1.2.4. Submódulos de Recursos Humanos (RH)

Abreviatura	Submódulo
RH	SEG Personal
	PLL Planillas

5.2. Nombre de clases, archivos o librerías

La definición del nombre de clases está definida por las siguientes reglas:

5.2.1. Cliente-Servidor (Power Builder) (Librería)

La librería debe estar definida según las siguientes reglas:

- Longitud total: 10 caracteres
- Formato: (issssmmmoa)
- Donde:
 - o is: Constante (Integrens)
 - o sss: Identificador de sistema
 - o mmm: Identificador de módulo
 - o oo: Identificador de objeto
 - wi: Window
 - dw: Data window
 - uo: User Object
 - mn: Menú
 - nv: Objetos no visuales
 - o a: Secuencia en relación al identificador del objeto en forma alfabética (a, b, c...)
- Ejemplos:
 - o La librería issysbasdwa será una librería de datawindows de uso común.
 - o La librería issysbaswia será una librería de windows de ancestros y de uso común.
 - o La librería issysbasuoa será una librería de objetos de usuario de uso común.
- En las librerías de uso común se almacenarán los objetos compartidos por los diferentes módulos.
- Restricciones:
 - o El cumplimiento de este estándar es obligatorio en todos los casos.

5.2.2. Web (C#, JavaScript, Typescript, Python, Java)

Clases: Está definido por:

- cls: Identifica una clase
- mmm: Identifica el módulo.

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	14 de 24
DACTA SAC			

- sss: Identifica un submódulo del sistema.
- uc_frm: prefijo que describe al formulario.
- aaaaaa: Identifica el nombre del formulario (6 caracteres).
- Ejemplo:
 - o cls_cmriped_uc_frm_pedido

Rutas NodeJS: Está definido por:

- rt: Identifica una ruta en nodejs con express.js
- mmmsss: Identifica el módulo y submódulo del sistema.
- Ejemplo:
 - o rt_cmriped.ts

Componentes React:

- La definición en el código de un componente se usará la nomenclatura Pascal Case, ejemplo: **FormularioLogin**, porque usar Snake Case y/o poner la letra parte inicial del componente con minúscula, generada problemas en el html.

5.2.3. Móvil (Java) (clase)

Está definido por:

- cls: Identifica una clase
- mmm: Identifica el módulo.
- sss: Identifica un submódulo del sistema.
- uc_frm: prefijo que describe al formulario.
- aaaaaa: Identifica el nombre del formulario (6 caracteres).
- Ejemplo:
 - o cls_cmriped_uc_frm_pedido

5.2.4. Movil (React TS native)

Clases:

- cls: Identifica una clase
- mmm: Identifica el módulo.
- sss: Identifica un submódulo del sistema.
- aaaaaa: Identifica el nombre del formulario (6 caracteres).
- Ejemplo:
 - o cls_cmriped_pedido

5.3. Nombre de variables

La definición del nombre de variables está guiada por las siguientes reglas:

- Utilizar la notación Snake.
- El nombramiento de la variable está dividido en dos partes. La primera parte corresponde a la combinación de los caracteres del alcance y del tipo de dato

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	15 de 24
DACTA SAC			

de la variable no mayor a 3 caracteres, y la segunda parte corresponde a un abreviado de la finalidad de la variable no mayor a 6 caracteres.

- Ejemplos:
 - o Variable local de tipo entero para la cantidad:
 - li_cantid
 - o Variable local de tipo double para el precio:
 - ldb_precio
 - o Variable global de tipo objeto de acceso al sistema:
 - gob_sisacc
 - o Variable local de tipo cadena para el flag de facturación electrónica:
 - ls_flagfe
 - o Variable local de tipo cadena para XML de la cabecera de pedido:
 - ls_cabped

5.4. Nombre de métodos o procedimientos

La definición del nombre de los métodos está guiada por las siguientes reglas:

- Utilizar la notación Snake.
- El nombramiento del método llevará la siguiente estructura separadas por guiones bajo: sp_sssmmm_acción_descripción
- Utilizar verbos o frases con verbos para identificar a los métodos (recuerde que los métodos declarados en una clase representan las acciones que realizarán los objetos creados a partir de dicha clase).
- El nombre los métodos deben decir lo que hace. No use nombres engañosos.
- Ejemplos:
 - o Método base para agregar un cliente:
 - sp_sysbas_ins_agregar_cliente
 - o Método del módulo de comercial para guardar pedido:
 - sp_cmriped_ins_guardar_pedido
 - o Método base para actualizar un producto:
 - sp_sysbas_upd_actualizar_producto

5.5. Nombre de funciones

La definición del nombre de los métodos está guiada por las siguientes reglas:

- Utilizar la notación Snake.
- El nombramiento de la función llevará la siguiente estructura separadas por guiones bajo:
 - o Si es una función de evento UI: eu_nombrecontrol_descripcion:
 - ue_txt_codcia_keypress
 - o Si es una función de evento de sistema ef_mmmsss_accion_descripcion:
 - fe_sysbas_ins_cliente
 - o Si es una función de un proceso dentro de la aplicación descripción:
 - uf_calcular_igv
 - uf_exportar_excel
 - uf_almac_al_ata

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	16 de 24
DACTA SAC			

- Utilizar verbos o frases con verbos para identificar a las funciones (recuerde que las funciones declaradas en una clase representan las acciones que realizarán los objetos creados a partir de dicha clase).
- El nombre de las funciones debe decir lo que hace. No use nombres engañosos.
- Ejemplos:
 - o Función base para consultar clientes:
 - uf_sysbas_con_consultar_cliente
 - o Función del módulo comercial para guardar pedido:
 - uf_cmriped_ins_guardar_pedido
 - o Función base para actualizar un producto:
 - uf_sysbas_act_actualizar_producto

5.6. Nombre de parámetros

Los parámetros son aquellas variables que reciben los valores enviados a los métodos y funciones que son considerados como variables locales. La definición del nombre de ellos está guiada por las siguientes reglas:

- Utilizar la notación Snake.
- El nombramiento de la variable está dividido en dos partes. La primera parte corresponde a la combinación de los caracteres del alcance y del tipo de dato de la variable no mayor a 3 caracteres, y la segunda parte corresponde a un abreviado de la finalidad de la variable no mayor a 6 caracteres. Estas dos partes deberán estar enlazadas por un único guión bajo.
- Ejemplos:
 - o Argumento de tipo cadena para el código de vendedor:
 - as_codven
 - o Argumento de tipo entero para el año:
 - ai_codano
 - o Argumento de tipo fecha para la fecha:
 - adt_fecemi
 - o Argumento de tipo decimal para el monto de depósito:
 - adc_mondep
 - o Argumento de tipo cadena para el XML de la cabecera del pedido:
 - as_cabped

5.7. Nombre de formularios, ventana, window

La definición del nombre de los formularios está guiada por las siguientes reglas:

- Utilizar la notación Snake.

5.7.1. Cliente-Servidor (Power Builder)

Ver documento **“Estandar programación Power Builder 12.5”**

5.7.2. Web (C#, JavaScript)

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
DACTA SAC		Página	17 de 24

- El nombramiento del formulario está dividido en dos partes. La primera parte corresponde a un prefijo que describe al formulario:
 - o uc_frm: Identifica para un formulario.
 - o uc_pup: Identifica para un Popup.
 - o uc_rpt: Identifica para un reporte.
 - o uc_tpt: Identifica un template.
- La segunda parte corresponde a un abreviado de la finalidad del formulario no mayor a 6 caracteres. Estas dos partes deberán estar enlazadas por un guión bajo.
- El formato que debe cumplirse es: uc_frm_nombre del formulario.
- Ejemplos:
 - o Mantenimiento de clientes: uc_frm_taclin
 - o Mantenimiento de productos: uc_frm_produc
 - o Popup de clientes: uc_pup_client
 - o Reporte de clientes: uc_rpt_client
 - o Template de clientes: uc_tpt_client

5.7.3. Móvil (Java)

- El nombramiento del formulario está dividido en dos partes. La primera parte corresponde a un prefijo que describe al formulario:
 - o uc_frm: Identifica para un formulario.
 - o uc_pup: Identifica para un Popup.
 - o uc_rpt: Identifica para un reporte.
 - o uc_tpt: Identifica un template.
- La segunda parte corresponde a un abreviado de la finalidad del formulario no mayor a 6 caracteres. Estas dos partes deberán estar enlazadas por un guión bajo.
- El formato que debe cumplirse es: uc_frm_nombre del formulario.
- Ejemplos:
 - o Mantenimiento de clientes: uc_frm_taclin
 - o Mantenimiento de productos: uc_frm_produc
 - o Popup de clientes: uc_pup_client
 - o Reporte de clientes: uc_rpt_client
 - o Template de clientes: uc_tpt_client

5.8. Nombre de controles (excluye componentes react)

- El nombre de los controles estará compuesto por un prefijo del tipo de control y del nombre abreviado del control separados por un único guión bajo. El tamaño del prefijo será de 3 caracteres y del nombre abreviado de 6 caracteres.
- La lista de prefijos para los controles será: prefijo_nombre del control.
- Ejemplos:
 - o Botón para aceptar: btn_acptar
 - o Identificador HTML para la fecha de emisión: ms_fecemi
 - o Identificador de tabla para la grilla de clientes: do_grvcli

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	18 de 24
DACTA SAC			

6. Convenciones de código

Estas convenciones de codificación tienen la finalidad siguiente:

Sobre declaración de variables

- Se debe evitar el uso de variables globales.
- Existen algunas variables de tipo global que sirven fundamentalmente para manejar los códigos de empresa, división y oficina.
- Las variables globales deben ser, en su mayoría, para configurar el proyecto. Estas variables deben declararse en los archivos. env, **web.config**, **gradle**, etc.

Parámetros

- Se utilizarán un objeto de instancia donde se pasarán parámetros entre objetos.

Limpieza de memoria

- Cuando se crea un objeto, no olvidarse de destruirlo ya que ocupa espacio de memoria.

Validación de datos

- Es muy importante validar los datos, ya que se cae en el mismo bucle de ingresar data que solamente bloquea el sistema.

Sobre la codificación

- Facilitar la lectura, modificación y mantenimiento del código.
- El indentado de los códigos deberá ser correcto a un TAB por nivel.
- Dar un aspecto coherente al código, de modo que los lectores pueden centrar su atención en el contenido y no en el diseño.
- Permitir a los lectores entender el código con más rapidez, ya que pueden hacer suposiciones en función de su experiencia anterior.
- Todos los controles HTML deben contar con su respectivo ID sin excepciones
- Los nombres de los CSS customizados, es decir; que no están definidos dentro del “**look and feel**” del producto, deberá indicar a que modulo, submódulo, ventana y control pertenecen.

6.1. Diseño del formato del código

En un buen diseño, el formato se usa para hacer hincapié en la estructura del código y facilitar la lectura. Debe de considerar las siguientes convenciones:

- Utilice sangría automática, o en su defecto una tabulación.
- Utilice una sola instrucción por línea.
- Utilizar solo una declaración por línea.
- Si a las líneas de continuación no se les aplica sangría automática, utilice una tabulación.
- Agregue al menos una línea en blanco entre las definiciones de métodos o funciones.
- Use una línea en blanco para separar un grupo lógico de código.
- Utilice paréntesis para que las cláusulas de una expresión sean evidentes, como se muestra en el siguiente ejemplo:

```

o If ( (li_cantid > li_candef) && (li_cantid <
  li_canmax) )
o {

```

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	19 de 24
DACTA SAC			

```

o  //
o  }

```

6.2. Comentarios

Los comentarios buenos y entendibles hacen el código más fácil de entender y mantener. Sin embargo:

- No escribas comentarios para cada línea de código o para cada variable declarada.
- Los comentarios deben estar al mismo nivel que el código (usar el mismo nivel de sangría).
- Utilice // para los comentarios.
- Escriba comentarios siempre que sea requerido. Pero un buen código requerirá mucho menos comentarios. Si todos los nombres de variables y métodos son entendibles, eso hará que el código sea muy legible y no necesitará muchos comentarios.
- No describa comentarios si el código es fácilmente entendible sin comentarios. La desventaja de tener muchos comentarios es que, si cambias el código y olvidas cambiar el comentario, esto provocará más confusión.
- Poca línea de comentarios hará el código más elegante. Pero si el código no es claro/legible y hay pocos comentarios, será peor.
- Si debes de usar una lógica **compleja** o **extraña** por cualquier razón, documéntala muy bien con suficientes comentarios.
- Realice una verificación ortográfica en los comentarios y asegúrese de usar una gramática y puntuación apropiada.
- El código fuente puesto entre comentarios, por fines de reemplazo de código, no tiene funcionalidad alguna dentro del sistema, por tanto, debe ser eliminado por completo del mismo, sólo en casos excepcionales.
- El objetivo final es, escribir código claro y legible de tal manera que no necesites ningún comentario para entenderlo.

7. Políticas para nuevos desarrollos

El desarrollo de las nuevas aplicaciones deberá contemplar la siguiente funcionalidad como mínimo:

- Multi país. Obligatorio.
- Multi idioma. Obligatorio.
- Multi Empresa. Obligatorio.
- Multi planta de producción. Según Módulo.
- Multi centro de distribución. Según Módulo.
- Multi Almacén. Según Módulo.
- Multi División de negocios. Obligatorio.
- Multi Sucursal. Obligatorio.
- Multi Moneda. Obligatorio.
- Multi usuario. Obligatorio.

7.1. Pruebas unitarias

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	20 de 24
DACTA SAC			

Se debe aplicar pruebas unitarias a aquellos procesos relevantes de la aplicación. Ejemplo: Si se tiene una función que calcula una serie de importes en base a muchos factores como el tipo de cambio, se debe realizar una prueba unitaria por cada caso o regla de negocio que se pueda presentar para dicha funcionalidad.

Para aquellas pruebas unitarias donde se manejen una serie de flujos que dependan del resultado de diversas condiciones, se debe usar los **mocks** y **spy** para verificar, por ejemplo, que una función anidada no se esté invocando cuando no le corresponde.

Es recomendable el uso de las pruebas unitarias por cada proyecto que se haga. Este servirá para garantizar que las funcionalidades ya establecidas no se rompan con cambios futuros.

7.2. Programación de Logs

Toda aplicación que se desarrollo debe registrar logs de cada acción relevante del sistema y almacenarlas dentro de la solución con formato. json o **.txt**

7.3. Documentación de los proyectos

Los proyectos deben tener 2 tipo de documentación:

- Manual de Uso: Esta documentación es un manual para el cliente o usuario final sobre cómo funciona la aplicación.
- Manual para DevOps: Este manual se realiza en el archivo Readme.md. En él se incluída información técnica sobre la aplicación. qué es lo que contiene, cómo configurarla, como desplegarla, credenciales, integraciones, etc.

7.4. Sobre el uso del Git para DevOps

- Consultar el documento **Manual y políticas de uso del GIT**.

8. Buenas prácticas de programación

- Evita escribir métodos y/o funciones muy largas. Un método debe típicamente tener entre 1 a 25 líneas de código. Si un método tiene más de 25 líneas de código, debes considerar refactorizarlo en métodos separados.
- El nombre de los métodos y/o funciones debe describirlo que hace. No se debe utilizar nombres engañosos. Si el nombre del método es obvio, no hay necesidad de documentación que explique qué hace el método.
- Un método debe tener una sola tarea. No se debe combinar más de una tarea en un solo método, aún si estas tareas son pequeñas.

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
DACTA SAC		Página	21 de 24

- Siempre verifique valores inesperados. Por ejemplo, si estás utilizando un parámetro con 2 posibles valores, nunca asumas que si uno no concuerda entonces la única posibilidad es otro valor.
- Convierte las cadenas de texto en minúsculas o mayúsculas antes de compararlas. Esto asegurará que la cadena coincida.
- Evita usar variables globales. Declara variables locales siempre que sea necesario y pásalas a otros métodos en vez de compartir una variable global entre métodos. Si compartes una variable global entre métodos, te será difícil qué método cambia el valor y cuando.
- Nunca incrustes en el código rutas o letras de dispositivos. Obtén la ruta de la aplicación programáticamente y usa rutas relativas a ella.
- No asumas que tu código se ejecutará desde el disco "C:". No puedes saber si algunos usuarios lo ejecutan desde la red o desde "Z:".
- Asegúrate que todos los archivos requeridos y las dependencias estén disponibles en las ubicaciones esperadas. Verifica las conexiones a la base de datos, si es requerido.
- Bríndale un mensaje amigable al usuario en caso de algún problema.
- Los mensajes de error deben ayudar al usuario a resolver el problema. Nunca muestres mensajes de error como "Error en la Aplicación", "Hay un error...", etc. En vez de ello, da mensajes específicos como "Falló al actualizar la base de datos debido a que la cuenta y/o la contraseña están incorrectos".
- Muestra mensajes cortos y amigables al usuario. Pero registra el error actual con toda la información posible. Esto ayudará mucho a diagnosticar problemas.
- No guardes más de una clase en un solo archivo.
- Evita tener archivos muy grandes. Si un solo archivo tiene más de 1000 líneas de código, es un buen candidato a refactorizar. Divídelos lógicamente en dos o más clases.
- Evita pasar muchos parámetros a un método. Si tienes más de 4~5 parámetros, es un buen candidato a definir un objeto con una estructura. Lo contrario destroza el consumo en memoria. Más facilidad de corrupción de datos, más castigo a los ciclos del procesador, etc.
- Si tienes un método y/o función que retorna una colección, devuelve una colección vacía en vez de "null", si no hay datos que retornar. Por ejemplo, si tienes un método que retorna un arreglo de datos, siempre retorna un arreglo válido. Si no tienes elementos que devolver, retorna entonces un arreglo con 0 elementos. Esto hará fácil para la aplicación que llama al método verificar solamente la propiedad "Count" en vez que hacer verificaciones adicionales para "null".
- Organiza lógicamente tus archivos dentro de carpetas apropiadas según la arquitectura de las carpetas.
- Los archivos que se envían a lado cliente como *.js, *.css y *.html deben estar minificados, y con más razón si estos pesan demasiado.
- Utilizar el tipo de variable correcto en typescript y evitar el uso de "any".
- Importar las librerías y lugares estratégicos que no afecten el rendimiento de la aplicación y solo invocar aquellas que serán usadas.
- Si bien los "console.log" y "debugger" ayudan a realizar debugs en las aplicaciones javascript, deben eliminar cuando se dejen de utilizar.

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	22 de 24
DACTA SAC			

- Declara variables tan cerca como sea posible de donde son usadas por primera vez. Usa una declaración de variable por línea.
- Siempre debe realizarse pruebas unitarias sobre las funciones que se programen.
- En la medida de lo posible definir las funciones como asíncronas.
- No incrustar información sensible como contraseñas dentro del código fuente.
- Evitar el uso de código dinámico para prevenir inyección de código.
- Configurar el CORS de las aplicaciones para que solo permita los dominios que solo harán uso de la aplicación.
- Los WebService deben estar protegidos con autenticación del tipo Bearer y/o autenticación de tipo básica.
- Siempre optimizar el código, ya sea reduciendo la cantidad de sentencias en la lógica, utilizando tipos de variables que ocupen poco espacio en memoria, etc.
- En javascript usar las instrucciones "const" cuando la variable no será modificada y "let" cuando su valor será posteriormente cambiado.

9. Manejo de Excepciones

El manejo de excepciones se realizará de acuerdo al nivel de la excepción de la siguiente manera:

T-SQL:

<INI><FIN>

10. Mensajes de alerta

Los mensajes deberán ser descritos como constantes en un módulo.

Tipo de mensaje	Descripción
Mensaje de inserción	Se registró correctamente.
Mensaje de modificación	Se actualizó correctamente.
Mensaje de eliminación	¿Desea eliminar el registro?
Mensaje de campo incompleto	Ingrese dato en el campo
Mensaje de procesamiento	Se procesó correctamente.

10.1. Cliente-Servidor (Power Buidar)

- Se debe tener en cuenta la siguiente variable del nombre del Sistema:
 - o `String vnomapl = "Comercial"//Nombre del sistema`
- Para el caso de los mensajes de confirmación, se ha determinado que solo se deben usar las siguientes funciones en la siguiente forma:
 - o `if MessageBox(vnomapl, "Seguro de guardar la información?", question!, yesno!, 2) = 2 then return`
- Para el caso de las alertas se diferencian tres casos:
 - o **Correcto:**
`MessageBox(vnomapl, "La operación fue grabada exitosamente.")`
 - o **Alerta:**

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
		Página	23 de 24
DACTA SAC			

```

    MessageBox(vnomapl, "La operación fue cancelada.",
    Exclamation!)

```

- o **Error:**

```

    MessageBox(vnomapl, "La operación fue cancelada.",
    StopSign!)

```

10.2. Web (JavaScript)

- Para el caso de los mensajes de confirmación, se ha determinado que solo se deben usar las siguientes funciones en la siguiente forma:
 - o `var funaccept = function () { };`
 - o `var funcancel = function () { };`
 - o `confirmacion("", "Seguro de guardar la imagen?", funaccept, funcancel, { closeOnConfirm: true, html: true });`
- Para el caso de las alertas se diferencian tres casos:
 - o **Correcto:**

```

    alerta("", "La operación fue cancelada.", { type: "success" });

```
 - o **Alerta:**

```

    alerta("", "La operación fue cancelada.", { type: "warning" });

```
 - o **Error:**

```

    alerta("", "La operación fue cancelada.", { type: "error" });

```

10.3. Móvil (Java)

- Para el caso de las alertas se emplearán para todos los casos:
 - o `Toast.makeText(context, "Mensaje personalizado.", Toast.LENGTH_LONG).show();`

11. Seguridad de la información

Consultar el documento **I-O-DES-002 Seguridad de aplicaciones** donde se detalla la manera de invocar servicios SOAP, ApiRest y la ejecución de aplicaciones Cliente Servidor donde se detalla el manejo de la seguridad de nuestras aplicaciones.

Para la codificación segura se está adoptando las prácticas de codificación segura según el informe de OWASP Top 10. Consultar documento **I-O-DES-033 Instructivo OWASP Top 10 (2021)**.

11.1. Normativa de Seguridad de la Información

- Se aconseja grabar por periodo 30 minutos, archivos creados y actualizados para no tener problemas de perdida de información.
- Prohibida la copia parcial o completa de código fuente de las aplicaciones.
- Se deberá realizar commits al repositorio GIT mínimo 1 vez al día.

	SISTEMA INTEGRADO DE GESTIÓN	Código	M-O-DES-001
	ESTANDARES DE PROGRAMACION	Versión	3.0
		Vigente desde	22/10/2025
DACTA SAC		Página	24 de 24

12. Control de cambios

Versión	Fecha	Descripción	Elaborado por	Revisado por	Aprobado por
1	30/06/2021	Emisión	Coordinador de Desarrollo	Gerente de Proyectos	Gerente General
2	12/05/2023	Actualización	Coordinador de Desarrollo	Gerente de Proyectos	Gerente General
3	22/10/2025	Actualización	Coordinador de Desarrollo	Gerente de Proyectos	Gerente General
Firmas de la versión vigente					
			A. Morales	S. Rafaile	G. Rafaile
Identificación de las modificaciones					
Versión	Descripción de cambios				
2	Se actualizó el nuevo logo de DACTA en el documento.				
3	Adopción de mejores practicas según el informe de OWASP Top 10.				

Comentado [AML1]: